Project Title: Unsupervised Brain Tumor Detection Using BiGAN

Authors:

Name	Student ID
Ashim Chandra Das	20201042
Rajesh Mojumder	24166017

Submission Date:

25 June 2026

Advanced Training on Semiconductor and ICT Technology

BRAC University

Table of Contents

1. Introduction

1.1 Brief Introduction

Brain tumor detection is a critical challenge in medical imaging. Supervised approaches require large labeled datasets of tumor images, which are expensive and time-consuming to obtain.This project builds an unsupervised anomaly detection system using a BiGAN (Bidirectional Generative Adversarial Network) trained exclusively on healthy brain MRI scans; no tumor images are needed during training. The selected domain is Medical Image Analysis: specifically, brain MRI tumor detection using unsupervised deep learning. The system learns the statistical distribution of healthy brain anatomy. At inference, images the model cannot reconstruct well are flagged as anomalous, indicating a potential tumor. Finally, the system is a four-stage ML pipeline (data ingestion, preprocessing, BiGAN training, evaluation) combined with a Streamlit prediction app served via Docker, providing real-time tumor detection with a visual heatmap showing the suspicious region.

1.2 Problem Statement

Manual analysis of brain MRI scans is time-intensive and prone to inter-observer variability. An automated system that can reliably flag suspicious regions supports radiologists and speeds up diagnosis.

- **Input:** A brain MRI image (JPG or PNG) uploaded by the user.
- **Output:** A binary prediction, Healthy or Tumor Detected, together with a pixel-level reconstruction error heatmap that highlights the anomalous region when a tumor is

suspected.

- **Target users:** Medical professionals, researchers, and diagnostic decision-support systems in clinical or research settings.
- **Clinical relevance:** Early detection of brain tumors significantly improves patient outcomes. The unsupervised approach removes the dependency on large labeled tumor datasets, making the system practical in data-scarce medical environments.

1.3 Objectives

- Build a BiGAN model trained exclusively on healthy MRI images for unsupervised brain tumor detection.
- Detects brain tumors as reconstruction anomalies, without any tumor supervision, using anomaly score = 0.7xMSE + 0.3xSSIM error.
- Track all experiments (hyperparameters, losses, metrics, artifacts) using MLflow with DagsHub remote tracking.
- Provide a Streamlit prediction app that accepts an MRI upload and returns a diagnosis with a visual tumor region heatmap overlay.
- Containerize the prediction app using Docker for reproducible, platform-independent deployment.

2. Dataset and Preparation

2.1 Dataset Description

Dataset: Brain MRI dataset sourced from Kaggle Dataset.

Total samples: 1901 JPEG images; 988 healthy brain MRIs and 913 tumor brain MRIs.

- **Classes:** Two; Healthy (label 0) and Tumor (label 1). Labels are used only during evaluation; the training set contains healthy images only.
- **Input type:** Grayscale MRI images, converted to 128x128 pixels and normalised to [0, 1] during preprocessing.
- **Target variable:** Binary anomaly label (Healthy/Tumor), determined at evaluation time by thresholding the reconstruction-based anomaly score.

2.2 Data Access

Stage 01 of the pipeline downloads the dataset automatically from a Google Drive link using gdown. The archive is extracted to data/BRAIN MRI/ and the zip file is deleted immediately after extraction.

Raw data is not committed to the GitHub repository. Only the download script (src/tumor_detection/components/data_ingestion.py) and the configured source URL (configs/config.yaml) are version-controlled.

2.3 Preprocessing

Stage 02 applies the following preprocessing steps to every image:

- Convert to grayscale (single channel).
- Resize to 128x128 pixels using PIL.
- Normalise pixel values to the range [0, 1] by dividing by 255.
- Split: 80% of healthy images (790) form the training set; the remaining 20% healthy (198) plus 20% of tumor images (183) form the test set. Tumor images are never used during training.
- Save as NumPy arrays: X_train.npy, X_test.npy, y_test.npy; in artifacts/data_transformation/.

3. Methodology

3.1 Course Techniques Used

The following course techniques were applied in this project (see Table 1 below for file-level mapping):

Technique	Where used
Unsupervised deep learning (BiGAN/GAN)	model/BIGAN.py, components/model_training.py

Technique	Where used
MLflow experiment tracking	components/model_training.py, components/model_evaluation.py
Dockerized prediction app	Dockerfile
Streamlit prediction UI	app.py
Anomaly detection evaluation (ROC, PR curve, confusion matrix)	components/model_evaluation.py

3.2 Model Architecture

The model is a BiGAN with three jointly trained components (defined in model/BIGAN.py).

Encoder: 4x Conv2D blocks (32 to 64 to 128 to 256 filters, 4x4 kernel, stride=2) with channel-wise attention after each block, followed by Dense(512) to Dense(128) to produce a 128-dimensional latent code.

Generator: Dense to Reshape(8,8,256), then 4x Conv2DTranspose blocks (256 to 128 to 64 to 32 to 16) with residual connections and attention, ending with Conv2D + sigmoid to output a 128x128x1 image.

Discriminator: dual-branch architecture (image branch via Conv2D, latent branch via Dense) concatenated and classified as real or fake (image, latent) pairs using binary cross-entropy.

Loss:

- **Discriminator:** binary cross-entropy.
- **BiGAN:** binary cross-entropy.
- **Reconstruction:** 0.6xMSE + 0.3xMAE + 0.1xSSIM.

Optimiser: Adam lr=5e-5 (discriminator, BiGAN), lr=1e-4 (reconstruction).

Hyperparameters: latent_dim=128, img_size=128, batch_size=32.

3.3 Training Strategy

Training uses a two-phase strategy on 790 healthy images (batch size 32).

Phase 1 Pre-training (150 epochs): only the reconstruction model (encoder + generator) is trained with the combined pixel-level loss. This bootstraps the generator to produce recognisable reconstructions before adversarial training begins.

Phase 2 Joint training (500 epochs): the discriminator is updated every 5 epochs; the BiGAN (encoder + generator + discriminator) is updated every epoch; the reconstruction model is updated 3x per epoch for stable convergence. After training, encoder.keras, generator.keras, and discriminator.keras are saved to both artifacts/model_training/ and trainedmodels/.

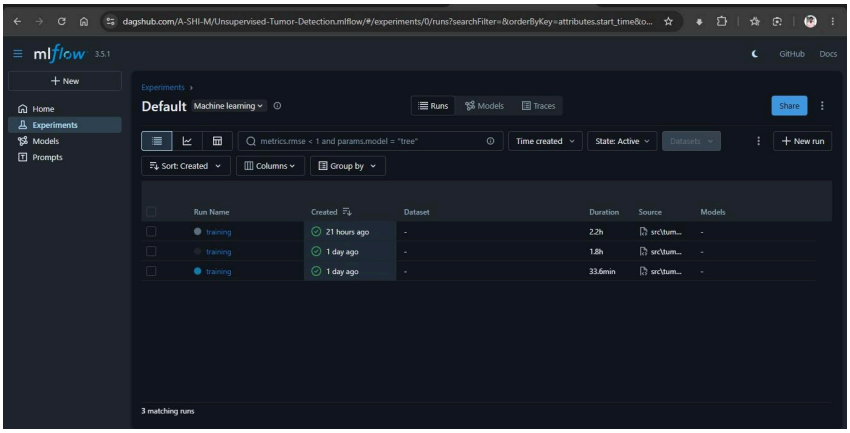
4. Experiments and Results

4.1 MLflow Tracking

All experiments are tracked on a DagsHub-hosted MLflow server (https://dagshub.com/A-SHI-M/Unsupervised-Tumor-Detection.mlflow). Training and evaluation are logged under a single MLflow run using a shared run_id saved to

trainedmodels/mlflow_run_id.txt.

- **Parameters logged:** epochs, pretrain_epochs, batch_size, img_size, latent_dim, lr_discriminator, lr_bigan, lr_reconstruction, perceptual_loss.
- **Metrics logged:** pretrain_recon_loss (every 10 pre-train epochs), d_loss, g_loss, recon_loss (every 10 joint-training epochs), roc_auc, accuracy, recall, specificity, f1_score, optimal_threshold.
- **Artifacts saved:** reconstruction progress images (every 20/50 epochs), encoder/generator/discriminator model files, loss curve plot, evaluation plots (ROC curve, PR curve, anomaly score distribution, confusion matrix heatmap, metrics summary bar chart), metrics.json.
- **Three training runs were logged:** Two with VGG16 perceptual loss and one with pixel-level loss only. The pixel-level run was selected as best because the unscaled VGG feature loss prevented pixel reconstruction learning.



4.2 Evaluation Results

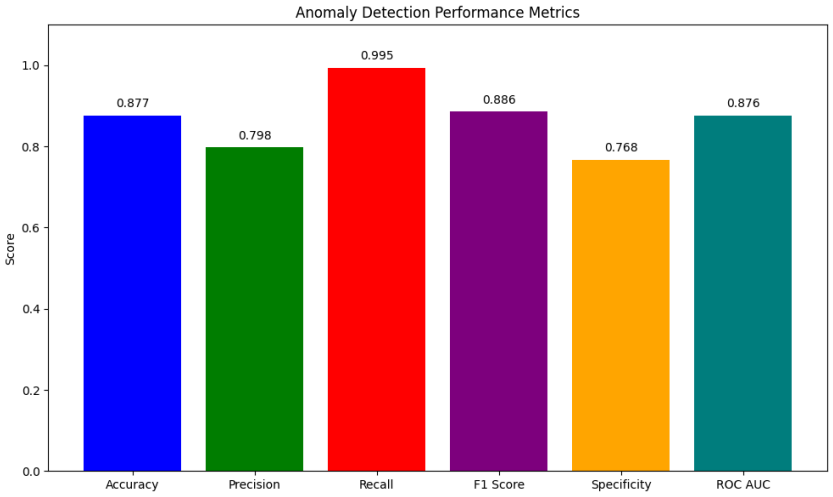
The retrained BiGAN (pixel-level loss, no VGG perceptual loss) was evaluated on 381 test images (198 healthy, 183 tumor) using reconstruction-based anomaly scoring (anomaly score = 0.7 x MSE + 0.3 x SSIM error). Results are summarised in the table below.

Confusion matrix: TN=152, FP=46, FN=1, TP=182. The model misses only 1 tumor out of 183, demonstrating high clinical sensitivity. The optimal threshold (0.1202) was determined by Youden's J statistic on the ROC curve.

Metric	Value
ROC AUC	0.876
Accuracy	87.7%
Recall (Sensitivity)	99.5%
Specificity	76.8%
F1 Score	0.886
Average Precision	0.782
False Positives	46 / 198 healthy images
False Negatives	1 / 183 tumor images

Metric	Value
Optimal Threshold	0.1202

Key finding: disabling the unscaled VGG16 perceptual loss (which dominated training at ~100,000x the scale of pixel MSE) and using only pixel-level loss raised ROC AUC from 0.652 to 0.876 and accuracy from 62.2% to 87.7%.



5. Prediction App and Docker

5.1 Prediction Pipeline

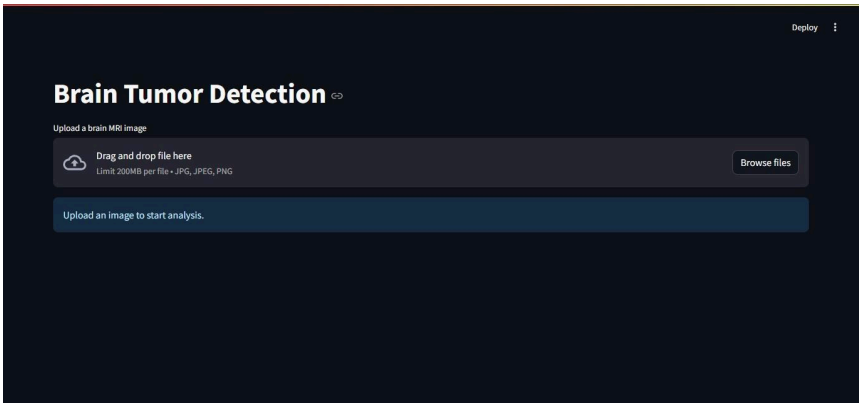
The prediction pipeline (app.py) operates as follows:

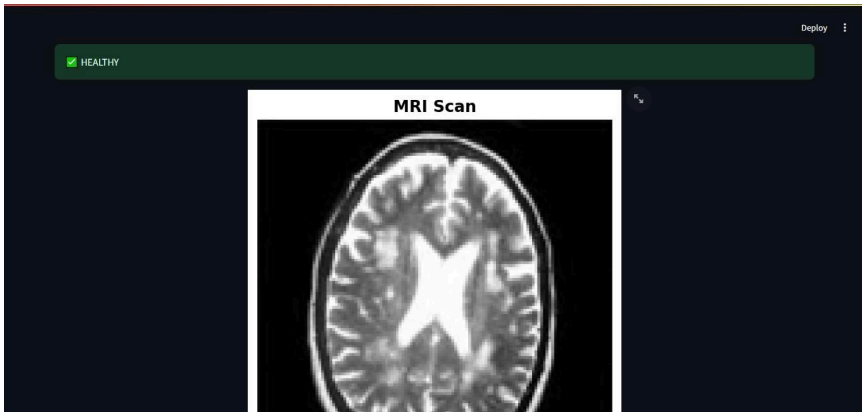
- The user uploads a brain MRI image (JPG or PNG) via the Streamlit file uploader.
- The image is converted to grayscale, resized to 128x128 pixels, and normalised to [0, 1].
- The trained encoder maps the image to a 128-dimensional latent code; the generator reconstructs the image from that code.

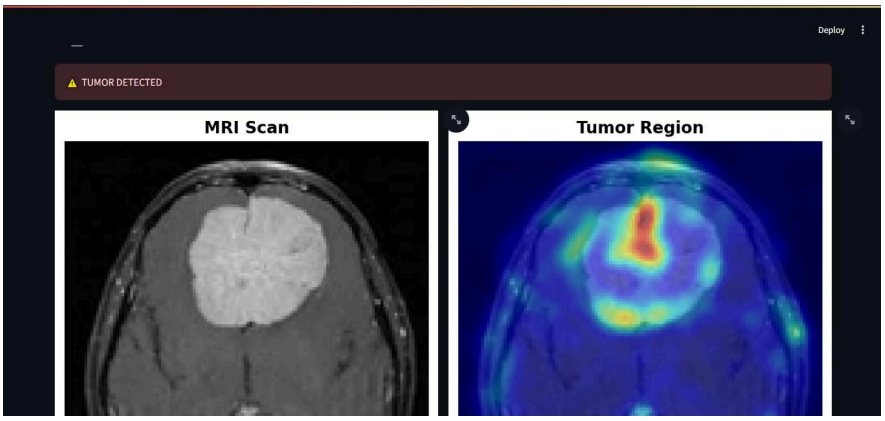
- Anomaly score = 0.7 x pixel MSE + 0.3 x SSIM error between the original and the reconstruction.
- If score >= 0.1202 (optimal threshold from metrics.json): Tumor Detected. For tumor predictions, a pixel-wise error map is computed, smoothed with a Gaussian filter (sigma=3), and overlaid on the original MRI as a jet-colourmap heatmap.

5.2 Prediction UI

The Streamlit app (app.py) shows a file uploader accepting JPG/PNG brain MRI images. After upload, inference runs automatically and the result is shown as a coloured banner: green (Healthy) or red (Tumor Detected). If Healthy: the original MRI image is displayed centred on the page. If Tumor: two panels are shown side by side — the original MRI (left) and the tumour region overlay (right), where a jet-colourmap heatmap (red/yellow = highest anomaly) is blended at 50% alpha on the MRI. The optimal threshold is loaded automatically from trainedmodels/metrics.json so the app always uses the latest calibrated value without code changes.

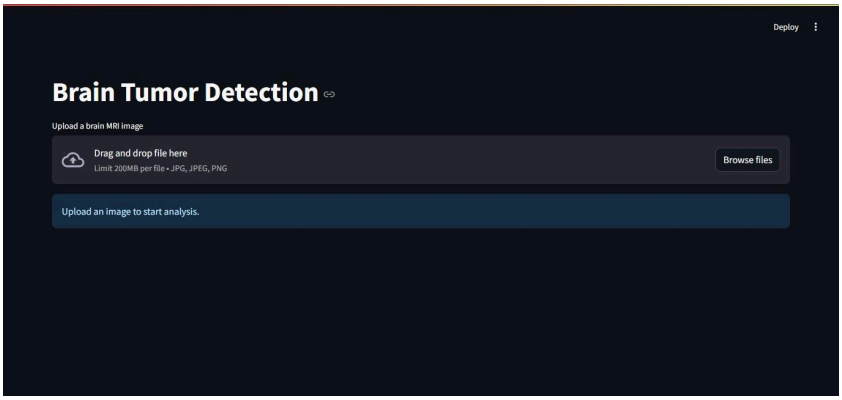






5.3 Docker Serving

- The prediction app is containerised using Docker (base image: python:3.12-slim, port 8501).
- The Dockerfile copies only the inference-required files: app.py, params.yaml, and the trainedmodels/ folder (models + metrics.json). Dependencies are installed from requirements.txt; the editable package install (-e .) is excluded since app.py does not import the tumor_detection package.
- **Build:** docker build -t tumor-detection:0.1 .
- **Run:** docker run -p 8501:8501 tumor-detection:0.1.
- The repository contains the Dockerfile and .dockerignore.



6. Repository and Reproducibility

6.1 Repository Structure

The repository is organised as follows:

- **src/tumor_detection/ :** pipeline package with four stages: data ingestion, data transformation, model training, model evaluation.
- **model/BIGAN.py:** BiGAN architecture (encoder, generator, discriminator).
- **configs/config.yaml:** pipeline paths.
- **params.yaml:** hyperparameters.
- **app.py:** Streamlit prediction UI
- **main.py**: full pipeline runner.
- Dockerfile, .dockerignore, requirements.txt, setup.py.

- **trainedmodels/:** encoder.keras, generator.keras, discriminator.keras, metrics.json, mlflow_run_id.txt.

6.2 GitHub Rules

The following are excluded from the repository via .gitignore:

- Raw dataset files (data/) - fetched at runtime by Stage 01.
- Virtual environment (venv/).
- Training artifacts (artifacts/) - generated by the pipeline.
- Credentials (.env file containing DagsHub token).

6.3 Reproducibility

A reviewer can reproduce the full pipeline from a fresh clone in the following order:

Step 1: Create Virtual Environment with python 3.12.10

Step 2: Install dependencies:

- pip install -r requirements.txt

Step 3: Get a DagsHub token:

- Create an account at dagshub.com.
- Go to User Settings -> Tokens -> Generate New Token.

Step 4: Create a .env file in the project root:

MLFLOW_TRACKING_URI=https://dagshub.com/A-SHI-M/Unsupervised-Tumor-Detection.mlflow

MLFLOW_TRACKING_USERNAME=dagshub-username

MLFLOW_TRACKING_PASSWORD=dagshub-token

Step 5: Run the full pipeline:

- python main.py (downloads data, transforms, trains BiGAN, evaluates)

Step 6: Inspect MLflow runs on DagsHub:

- https://dagshub.com/A-SHI-M/Unsupervised-Tumor-Detection.mlflow/

Step 7: Launch the prediction UI:

- streamlit run app.py

Step 8: Build and run the Docker container:

- Build Image: docker build -t tumor-detection:0.1
- Run Container: docker run -p 8501:8501 tumor-detection:0.1

7. Limitations and Future Work

7.1 Limitations

Small training dataset: only 790 healthy MRI images were used for training. A larger dataset would tighten the model's healthy distribution and reduce false positives.

Remaining false positives: 46 out of 198 healthy images are incorrectly flagged as tumors (~23%). Clinical deployment would require further calibration.

Binary classification only: the system outputs Healthy or Tumor without indicating tumor type, grade, or precise anatomical location.

Threshold sensitivity: the optimal threshold was calibrated on the current test split; it may need adjustment for images from different MRI scanners or acquisition protocols.

7.2 Future Improvements

Data augmentation: apply horizontal/vertical flips, rotations, and brightness jitter to expand the 790 training images without additional data collection.

Semi-supervised learning: incorporate a small number of labelled tumour images to tighten the decision boundary.

GAN-based data augmentation: use a conditional GAN to synthesise additional healthy MRI variants for training.

Improved anomaly localisation: replace the pixel-MSE heatmap with Grad-CAM or attention map visualisation for more interpretable region highlighting.

Clinical threshold calibration: validate the optimal threshold against radiologist-labelled ground truth from diverse MRI acquisition sources.

8. Conclusion

This project successfully built an unsupervised brain tumor detection system using a BiGAN trained exclusively on 790 healthy brain MRI images. Without ever observing tumor images during training, the final model achieves a ROC AUC of 0.876 and a recall of 99.5%, correctly identifying 182 out of 183 tumour cases (missing only 1). The system is deployed as a Streamlit web application containerised with Docker, providing real-time predictions with a visual tumour region heatmap generated from reconstruction error. A key experimental finding was that standard pixel-level reconstruction loss (0.6xMSE + 0.3xMAE + 0.1xSSIM) significantly outperforms VGG16 perceptual loss for this task. Unscaled VGG feature differences dominated the gradient at ~100,000x the scale of pixel MSE, preventing the model from learning pixel-level reconstruction quality. The team gained practical experience in unsupervised deep learning, GAN training dynamics, MLflow experiment tracking, and production-ready ML deployment with Docker and Streamlit.